FailBound-Lab

沙箱执行记录

跨环境复现证据:五个会话、16 项逐项对照

赛道 04 AI Agent · 2026-10-23

5	8640	16/16
沙箱会话(四组全量实验 + 一次端到端演示)	沙箱内独立重跑的 run 数	跨环境逐项对照,全部一致(不含样本外;S3 样本外重跑已中止,见 §7.3)

> Replay 链接(四件套第 ② 项「可运行作品链接」):
> https://sciencepal.ai/share/944c87a6-46b9-49b0-b49f-0adc67257c21
> 该链接是 SciencePal 上的一个共享 Agent 会话,thread 为 `944c87a6-46b9-49b0-b49f-0adc67257c21`,
> 时间 2026-10-18 18:21:19 → 22:14:00 UTC(北京时间 10-19 02:21 → 06:14)。全程单个 run
> `dcca8c8e-d8e2-451b-882e-57bfd8689e04`,无重启、无打断,公开可访问、无需登录。
> 会话记录的内容是:把提交版代码包上传进沙箱,完整执行四组全量实验(各 2160 run),
> 逐组验收行数、run 数、max spent 与 SHA256,并追加一次端到端演示。
> 2026-10-19 复核可正常访问,见第六节。
> 版本更正(2026-10-19):本文档此前描述的首轮沙箱会话 `1006bac1…` 不再作为第 ② 项提交。
> 那个会话跑的是 67 文件的早期快照包,且会话内含过程痕迹。首轮会话的结果作为历史记录
> 保留在第一至五节,第六节是用提交版代码包重跑的会话,以及它与原结果的逐行比对。

一、这个 Replay 里有什么(按时间顺序)

阶段	时间(UTC)	内容
① 上传与完整性门禁	10-16 07:09–07:16	上传 `failbound_lab_upload.zip` + `SHA256SUMS.txt`,解压后校验 31 个文件全部 OK
② 定位并修复一个真实 bug	10-16 07:16–07:17	`config.py` 的 `_PROJECT` 用 `../..` 上跳两级,在沙箱布局(`agent_lab/` 与 `data/` 平级于 `/workspace`)下解析到 `/`,`discover_tasks` 因此返回 0 个任务。改为上跳一级后恢复 18 个任务
③ 端到端小规模验证	10-16 07:24–08:25	`run_experiments.py gate 2`:144 run / 146 行 / 3641 秒,无报错,只有良性的 `ConvergenceWarning`。实测单 run 约 25.3 秒,据此推算全量 2160 run 需约 15 小时,原先「约 2 小时」的估计被推翻
④ 换包 + 验收测试	10-16 17:03–17:08	换用新包 `failbound_full_ablate_upload.zip`(67 个文件 = 13 个 `agent_lab/` + 53 个 `data/`,SHA256 `69c79a06…`)。该包的 `config.py` 已是修复后的动态版,沙箱无需再改。验收 `--max-budget 200` 确实生效(8 run / 10 行输出)。此包为早期快照:数值核心自包含(`run_experiments.py` 内联主动学习循环),但不含后来重构出的 `trial.py` / `failbound_agent.py` / `out_guard.py` 等 12 个脚本、8 个 agent 配置导出与 9 个 scan 数据,见第六节
⑤ 四组全量执行	10-16 17:08 → 10-17 11:41	见下表
⑥ 交付	10-17 11:41 → 12:25	汇总 4 组结果、计算 SHA256、打包交付

二、四组执行结果(沙箱侧,预算上限 200)

文件	行数	run 数	task_meta	arm	gate	max spent	耗时	判据
runs_nogate.jsonl	2179	2160	18	main	False	**200.0**	4686s(≈78 min)	通过
runs_gate.jsonl	2179	2160	18	main	True	**205.0**	2251s(≈38 min)	通过(≤205)
runs_nogate_ablate.jsonl	2179	2160	18	ablate	False	**200.0**	3786s(≈63 min)	通过
runs_gate_ablate.jsonl	2179	2160	18	ablate	True	**204.0**	2138s(≈36 min)	通过(≤205)

行数构成恒为 2179 = 1 meta + 2160 run + 18 task_meta,与本地全量数据(预算 400)逐结构一致。

门控组 spent 的上界可以算出来。单候选成本最大为 6(`n_atoms>40` ×2 × 含贵金属 ×3),循环条件是 `while spent < budget`,故上界 = 预算档 + 5。沙箱 gate main 实测 max 205.0、gate ablate 实测 max 204.0;本地全量(预算 400)实测 max 405.0。三处实测落在同一条公式上,说明这是成本模型的必然结果,不是 bug。该规则对四种策略完全对称,没有方向性偏差。

三、这为什么构成跨环境复现检验

预注册 §10 登记过运行环境分级:沙箱跑预算 200 × 30 种子,本地跑全档 400 × 30 种子,两环境用同一套代码、同一种子派生规则(blake2b)、同一统计口径。

原定的检验方式是:沙箱的 4 个 jsonl 回传本地后执行 `compare_environments.py`,逐 run 比对 chosen 选择序列。`run_one` 在给定(任务、策略、种子、数据)之后完全确定性,序列一致即说明代码与数据在两环境逐比特相同。只比汇总统计量,会漏掉数字碰巧一样但路径不同的情况。

我们走的正是这条路,并测出一个修正:上述完全确定性要加上同一浮点环境的前提。换库版本后约六成 run 的轨迹会因近平局决策翻转,定量数据见 §5.5。这不削弱本检验的效力(代码或数据若不同,翻转的会是全部而非六成,且组级统计量与文件级极值指纹仍高度稳定),但逐比特一致的承诺只对 `random_uniform` 这类没有浮点决策路径的策略成立。

四、需要说明的三点

1. 沙箱执行跨越约 18.5 小时(10-16 17:08 → 10-17 11:41),因为沙箱单 run 约 25 秒,且 GP 开销随已观测数三次方增长。这不是一次跑完,而是四组顺序执行,每组都有独立的完成验证。
2. Replay 末尾(12:18–12:25)有一段重复的打包尝试,是沙箱 Agent 在打包环节陷入的循环,不影响已完成的四组数据。该段原样保留,未做剪辑。
3. 数据交付方式:应我们的要求,沙箱没有暴露 8081 端口,4 个 jsonl 原计划在对话中交付。原因是端口链接依赖沙箱会话存活,而本作品的一条核心主张是可复现不应依赖某个平台还活着,用一个会失效的端口链接交付数据与这条主张矛盾。

五、沙箱文件未回传后的证据恢复(2026-10-17 晚追加)

> **2026-10-21 补充说明(与本节相关,先读这一句)**:技术报告 §3.3.1 与 §3.4 的两项分析(同维对照、预算去向)**不在沙箱里跑**。它们是包外脚本 `matched_dim_analysis.py`,只读 `results/summary.csv`、`results/runs_nogate_ablate.jsonl` 与 `data/ablate/*.csv`,不产生新 run、不改任何既有产物。因此提交包 `b6d0a514…` 的 101/101 逐比特一致性不受影响,本记录里所有沙箱侧数字也不受其影响。

四组数据跑完并验证通过后,4 个 jsonl 的文件本体最终没有回到本地。沙箱按约定选择在对话里交付,但交付消息中只有每个文件的 SHA256 与聚合数字(SciencePal 的 storage 桶为空,附件从未真正上传)。`compare_environments.py` 因此一直没有可比的沙箱侧数据。本节记录在此基础上实际做到的事。

5.1 Replay 全量恢复(anon key 轮换后的重新提取)

SciencePal 前端在 10-17 轮换了 Supabase anon key,旧 key 已失效(`Invalid API key`)。重新提取的方式是:抓取分享页 HTML,提取其引用的 54 个前端 JS chunk,在 `main-app-*.js` 中找到 `NEXT_PUBLIC_SUPABASE_URL` 与 `NEXT_PUBLIC_SUPABASE_ANON_KEY`(新 key,exp 2098 年)。用新 key 分页拉取全部 1895 条消息到本地,恢复了对 Replay 的完整读取能力。评审期内可用同一方法复核可访问性。

5.2 从 Replay 中提取的沙箱承诺物

全部来自沙箱自己的命令输出(未截断的 tool output),已固化到 `results/sandbox_claims.json`:

承诺物	值	Replay 出处
4 个 jsonl 的 SHA256	`816326b7…` / `1b1e2010…` / `cd7b3434…` / `d056ae2a…`	12:15:32 `sha256sum` 原始输出 + 12:17:25 交付消息
沙箱所跑包 SHA256	`69c79a06826ce7bd…`(与上传一致,沙箱已自查;注:这是 67 文件早期快照包,提交包 `b6d0a514` 的重跑见第六节)	12:15:32 + 12:17:25
4 个 Replay 日志 SHA256	`9c186ec8…` / `e8b8ce74…` / `5d997d07…` / `b0e77a59…`	12:15:32
逐组 max_spent / 行数 / run 数	205.0 / 200.0 / 204.0 / 200.0,各 2179 行 / 2160 run / 18 task_meta	11:40:47 沙箱验证脚本输出
20 个任务的 `[done]` 行(n_pool + thr,共 26 个门控/无门控观测点)	见 `sandbox_claims.json`	各组执行日志
环境库版本	numpy 2.3.0 / pandas 2.3.3 / scipy 1.15.3 / sklearn 1.8.0(Python 3.11)	07:17:47

5.3 本地数据的一处修复(合并残留)

核对时发现本地 `runs_nogate.jsonl` 只有 2162 行(1 meta + 2160 run + 1 task_meta),比沙箱结构少 17 行 task_meta。这是 10/17 数据事故恢复时的合并残留,唯一幸存的一行被追加到了文件末尾,内容还是第一个任务的。run 数据 2160 条完整无损,缺的只是元信息头。

修复方式:`repair_task_meta.py` 按 `run_experiments.py` 写入时的同一算法再生 task_meta 行。再生算法先在三个完整文件(gate / gate_ablate / nogate_ablate)的 54 行现存 task_meta 上逐字节验证通过,再补齐 nogate 缺失的 17 行(每任务 120 run 后插一条)。修复后 4 个本地文件均为 2179 行,2160 条 run 逐字节未动(有断言保护),原文件已备份为 `runs_nogate.jsonl.bak-*`。

5.4 五层跨环境验证(`verify_sandbox_claims.py`,无需沙箱文件)

层	内容	方法	结果
L1	代码一致	提交清单(101 项)≡ 本地当前文件(打包自洽检查,见正文)	通过 101/101
L2	数据一致	同上(清单含全部 `data/*.csv`)	是
L3	任务参数一致	本地复算 n_pool/thr ≡ 沙箱 `[done]` 行 26 个观测点	通过 26/26
L4	结构一致	本地 4 文件均 2179 行 / 2160 run / 18 task_meta / 2160 唯一键	是
L5	行为指纹一致	本地 400 档 chosen 按沙箱循环(`while spent < 200`)截断,逐组 max spent ≡ 沙箱承诺	3/4 组精确一致;gate_ablate 205.0 vs 204.0(见 5.5)

L1/L2 的说明(10-18 复核后的校正):本层原脚本 `verify_sandbox_claims.py` 实际做的是「提交清单 `SHA256SUMS_full_ablate.txt`(101 项)≡ 本地当前工作树文件」的打包自洽核对。它证明冻结的提交 zip 与本地源码一致(101/101),但并未直接打开沙箱所跑的包做比对。10-18 发现沙箱实际跑的是 67 文件早期快照包(`69c79a06…`),与提交包(102 文件 `b6d0a514…`)逐文件比对后确认:67 个共有文件中 58 个字节完全相同、9 个有差异(差异文件为 `README.md` / `analyze.py` / `build_scan.py` /

`check_baseline_stability.py` / `config.py` / `merge_shards.py` / `run_experiments.py` / `scan_manifest.csv` / `preregistration.md`;其中 `config.py`、`run_experiments.py` 各差 24/-10 行,均属非数值路径,差异落在文档、分析、构建脚本与注释结构层,不影响 run 输出)。因此本层的密码学级结论应降级理解为打包自洽加子集一致性。沙箱执行的包等于提交的包,这条硬证据在第六节:干净会话 `944c87a6…` 在沙箱内校验 `6698192` / `b6d0a514…`(沙箱根目录 zip 实测 6,698,192 字节,与提交包逐字节对上),且其四组结果与早期快照包的结果除 meta `created` 时间戳外逐比特相同。这既证明提交包被如实执行,也证明重构(67→102 文件)数值精确等价。

L5 的意义与边界:max spent 是成本轨迹的指纹。若两侧代码或数据不同,chosen 序列会大规模改变,该值几乎必然漂移,所以它能排除实质性差异。但它不是逐比特指纹:即使代码数据逐比特相同,浮点环境(库版本、BLAS 线程)的差异也会让少量近平局决策翻转。我们后来用全组定量实验(§5.5)把这种环境级敏感性的粒度测到了个位数百分比:换库版本时约六成 run 的轨迹会翻转,但组级统计量差异 ≤2.4%,文件级 max 只在 ±1 个成本单位的带内移动。另需说明,nogate 两组的成本被压平成 1.0(等成本口径,`cost_model.cost_vector`),每个 run 必然恰好花光 200.0,所以这两行的一致属于结构性恒等,不含轨迹信息;L5 的行为信号全在 gate 两组。

5.5 跨环境复现的真实粒度:全组定量实验

L5 中 gate_ablate 组本地截断 max spent = 205.0,沙箱承诺 204.0。本地该组截断值的完整分布(2160 run,成本为 1–6 整数):

截断值	200.0	201.0	202.0	203.0	204.0	205.0
run 数	1605	294	255	2	1	3

达到 205.0 的恰好 3 个 run,全是 BO 策略:`matbench_dielectric__pca2/gp_ei/9`、`matbench_expt_gap__rp2/gp_ucb/17`、`matbench_log_gvrh__elem20/gp_ei/9`。沙箱该组最高 204.0,说明这 3 个 run 在沙箱都没达到 205.0。

差异只有 1 个成本单位,但根因和粒度都做了实测。在沙箱同库版本栈(sbstack venv:numpy 2.3.0 / scipy 1.15.3 / sklearn 1.8.0 / pandas 2.3.3,Python 与本地同为 3.13,BLAS 线程数与生产一致控在 1)下重跑两个 gate 组全量(`sbstack_probe.py`,各 2160 run),与本地默认栈逐 run 比对 chosen 前缀:

组	轨迹翻转	farthest_point	gp_ei	gp_ucb	random_uniform
gate	**1324/2160(61.3%)**	509/540(94%)	404/540(75%)	411/540(76%)	**0/540(0%)**
gate_ablate	**1247/2160(57.7%)**	390/540(72%)	427/540(79%)	430/540(80%)	**0/540(0%)**

有三点事实需要写清:

1. `random_uniform` 是唯一 100% 跨环境稳定的策略(540/540 逐点一致)。它的决策路径是纯 RNG 整数抽样,没有任何浮点运算,这从机理上解释了它为什么零翻转。
2. `farthest_point` 与 BO 的轨迹对库版本高度敏感(72%–94% 翻转)。机理由 `compare_trajectories.py` 逐点定位得到:轨迹前若干步完全一致,随后单个近平局决策被浮点差异打破(BO 是两个候选的 EI/UCB 后验值极接近,farthest_point 是两个候选的最大最小距离极接近),此后训练集与已选集合不同,后续选择蝴蝶效应式发散。分歧之后两环境 chosen 集合的交集只剩 4–5 个点。
3. 翻转不等于结论翻转。翻转的 run 里,截断 spent 完全相同的占 67%–71%,差 1 个单位的约 16%–20%,差 2 个的约 11%–13%,差 ≥3 个的仅 6/2571 个 run(成本是 1–6 整数,截断值落在 [200, 205] 带内)。

文件级 max spent 的三环境对照(C 列为沙箱承诺值):

组	A 本地默认栈	B sbstack(沙箱库版本,Py3.13)	C 沙箱(B 库版本,Py3.11)
gate	205.0	204.0	**205.0**
gate_ablate	205.0	**204.0**	**204.0**

max spent 是 2160 个 run 上的极值统计量。约六成轨迹翻转后分布顶部重新洗牌,它只在 ±1 个成本单位的带内移动。沙箱的两个承诺值都落在这个带内,gate_ablate 的 204.0 还被 B 环境精确复现。它能排除代码与数据的实质性差异(那会让整个分布变形、max 大幅漂移),但不是逐比特指纹,这一点已在上文写明。

线程数也是浮点环境变量,补充登记在此:同一台机器、同一套库,8 线程对生产单线程可使个别 GP run 翻转(抽样 44 个 run 中 1 个,`log_gvrh__elem20/gp_ei/7`)。生产用单线程(meta 里 `n_proc` 由 6 降为 1,即 1 线程每进程),sbstack 探针同样单线程,故上表的 A/B 比对已控住线程变量。早前「线程数已排除」的说法只基于 3 个 run,样本不足,现修正为:线程敏感性存在但稀有(约 2% 量级),库版本敏感性常见(约 60% 量级)。

组级统计量的稳健性(`compare_aggregates.py` / `compare_hits.py`,budget-200 口径):约六成轨迹翻转的前提下,组级均值几乎不动。

组	策略	n_exp 差	spent 差	hits 差
gate	farthest_point	+0.01%	+0.003%	−0.18%
gate	gp_ei	−0.81%	+0.042%	−1.70%
gate	gp_ucb	+0.51%	+0.016%	+0.28%
gate_ablate	farthest_point	+0.01%	−0.007%	+0.04%
gate_ablate	gp_ei	+0.39%	+0.006%	+0.62%
gate_ablate	gp_ucb	−0.13%	−0.017%	−2.39%
两组	random_uniform	**0.00%**	**0.000%**	**0.00%**

BO 增益(vs farthest_point)的符号与排序在两栈下完全保持:gate 的 gp_ucb +9.49→+9.60、gp_ei +8.47→+8.05,gate_ablate 的 gp_ucb +4.89→+4.20、gp_ei +3.24→+3.39,均远超噪声。

影响评估:

- 该分歧不触及任何已提交的科学结论。沙箱文件从未参与分析(分析部分在本地做是既定约定),本地分析全部使用预算 400 的全量数据;上面的组级对比进一步表明,即使换成另一套合法浮点环境重跑,结论统计量的变化也在千分位到 2.4% 以内。
- 跨环境复现的真实粒度现在是实测值:RNG 策略 100% 稳定,FP 敏感策略约四成轨迹稳定、六成翻转但截断行为大多不变(67%–71% 完全相同),组级统计量差异 ≤2.4%,文件级极值指纹在 ±1 成本单位带内。
- 若沙箱文件日后回传,`compare_environments.py` 的预期结果已有数字支撑:绝大多数 run 存在浮点级路径分歧(约六成),`random_uniform` 逐比特一致,聚合统计量吻合。这就是跨环境复现在这个作品里的实际粒度,我们选择定量登记。

5.6 证据链小结

#	证据链环节	对应验证层 / 结论
1	`pack_upload.py` 打包时逐文件算 SHA256 写入清单	打包自洽
2	上传包 SHA256 = `69c79a06…`(67 文件早期快照),沙箱自查与上传一致	传输完整性
3	沙箱解压执行四组全量(预算 200 × 30 种子 × 18 任务 × 4 策略)	真实执行
4	交付承诺:4 个文件 SHA256 + 聚合数字 + `[done]` 观测(现存于 Replay)	可外部核对
5	本地文件 ≡ 清单 67/67	L1/L2(密码学级,覆盖该包全部文件)
6	本地 runs 结构 ≡ 沙箱承诺	L4
7	本地截断行为指纹 ≡ 沙箱承诺(3/4 组精确,1 组差 1 成本单位)	L5
8	沙箱同库版本栈全组重跑(`sbstack_probe.py`,2160×2)	翻转率、机理、组级稳健性全部定量(§5.5)

#	证据链环节	对应验证层 / 结论
9	根因 = 浮点环境级近平局翻转,非代码或数据	§5.5 结论
10	发现该包只是提交包(102 文件 / `b6d0a514…`)的严格子集	版本门控漏洞(10-18 复核发现)
11	用提交包重开干净会话 `944c87a6` 完整重跑四组	提交包级覆盖
12	结果除 meta `created` 时间戳外与本节四组逐比特相同	第六节

结论分两层。其一,提交 zip 与本地工作树打包自洽(§5.4 L1/L2,101/101);其二,干净会话 `944c87a6…` 在沙箱内实测执行的就是提交包(根目录 zip 6,698,192 字节、`b6d0a514…`),且其结果与早期快照包结果除 meta 时间戳外逐比特相同(§6),即沙箱执行的包等于提交的包,重构未改数值。行为指纹 3/4 组精确一致、1 组差 1 个成本单位。

分歧的根因与粒度已用全组定量实验测得并登记(§5.5):换库版本时约六成 run 的轨迹因近平局决策翻转,但 `random_uniform` 100% 稳定、翻转 run 的截断行为 67%–71% 不变、组级统计量差异 ≤2.4%、文件级极值指纹只在 ±1 成本单位带内移动。沙箱两个承诺值均落于带内,其中一个被沙箱同库版本栈精确复现。

六、用提交包重跑(944c87a6)与等价性实证(2026-10-18/19)

6.1 为什么要重跑

第五节的 L1/L2 验证(针对 `69c79a06…`,67 文件)只能证明沙箱旧包与本地对应文件一致。10-18 复核时发现一个版本门控漏洞:冻结并跑过沙箱的那个包 `failbound_full_ablate_upload.zip`(SHA256 `69c79a06…`,5,469,519 字节,67 文件 = 13 个 `agent_lab/` + 53 个 `data/` + `preregistration.md`)只是提交版代码包 `failbound_full_ablate_upload.zip`(SHA256 `b6d0a514…`,6,698,192 字节,102 文件 = 33 个 `agent_lab/` + 62 个 `data/` + `preregistration.md` + 其余新增文件)的严格子集,后者多出 12 个脚本(`trial.py` / `failbound_agent.py` / `out_guard.py` / `verify_trial.py` / `audit_decision_card.py` / `check_report_consistency.py` / `compare_environments.py` / `export_agent_config.py` / `merge_runs.py` / `md_to_pdf.py` / `run_new_tasks.py` / `screening_demo.py`)、8 个 agent 配置导出与 9 个 scan 数据(差异清单按 102 文件口径需重新核验,原 12+8+9=29 项仅覆盖部分新增文件)。

这动摇了第 ② 项(可运行作品链接)的叙事根基:原提交候选 `1006bac1…` 跑的是 67 文件旧包,与最终要交的 102 文件包不是同一个。

处置方式(用户决策,2026-10-18):用提交包重开干净会话完整重跑,使沙箱执行的包等于提交的包。

6.2 重跑会话 944c87a6(第 ② 项链接本体)

项目	值
thread	`944c87a6-46b9-49b0-b49f-0adc67257c21`
唯一 run	`dcca8c8e-d8e2-451b-882e-57bfd8689e04`(单 run,无重启、无打断)
起止(UTC)	2026-10-18 18:21:19 → 22:14:00(北京时间 10-19 02:21 → 06:14,约 3h53m)
上传包	`failbound_full_ablate_upload.zip`,**6,698,192 字节**,SHA256 `b6d0a514…`(102 文件);沙箱内校验 `6698192` / `b6d0a514…` 一致,沙箱文件清单实测 zip 大小 6,698,192,与本地提交包逐字节对上
四组结果	各 **2179 行 / 2160 run / 18 task_meta**;max_spent **200.0 / 205.0 / 200.0 / 204.0**(全 ≤205 已核)
第四步 demo	`screening_demo.py --budget 200 --seeds 10 --counterfactual` 成功
文件回传	4 个 jsonl 本体已拉回本地(`sciencepal_files/runs_*.jsonl`),首次真正拿到文件而非仅哈希

> 沙箱文件清单实测:`944c87a6` 沙箱(id `7f61ad72…`)根目录 zip 大小 6,698,192;对照 `1006bac1` 沙箱(id `488ebf26…`)与 `d415822e` 沙箱(id `929e07c1…`)根目录 zip 大小均为 5,469,519。三者包字节数的差异本身就是提交包区别于旧包的客观物证。

6.3 等价性实证(重构精确等价 + 跨会话确定性)

为证明重构(67→102 文件)没有改数值,把 944c87a6(提交包)的四组结果与同四组用早期快照包跑出的结果逐文件比对:

- 早期快照包下的文件本体取自 10-18 的取回会话 `d415822e…`(旧包 `69c79a06`,沙箱 zip 5,469,519 字节;该会话中途被误判僵死中断一次,已恢复跑完,最终 4 组完整),存于 `old_pkg_results/runs_*.jsonl`;
- 提交包结果存于 `sciencepal_files/runs_*.jsonl`。

逐文件比对(脚本 `compare_runs.py`):

文件	行数	除 meta 行外是否逐比特相同	组级 total_nhits	组级 mean_hits
`runs_nogate.jsonl`	2179	是(仅 `created` 时间戳不同)	70635 / 70635	32.701389 / 32.701389
`runs_gate.jsonl`	2179	是	46250 / 46250	21.412037 / 21.412037
`runs_nogate_ablate.jsonl`	2179	是	67277 / 67277	31.146759 / 31.146759
`runs_gate_ablate.jsonl`	2179	是	50759 / 50759	23.499537 / 23.499537

(左列为旧包结果、右列为提交包结果。各 2179 行(1 条 meta + 2160 条 run + 18 条 task_meta)完全对齐,其中纯 run 行 2160 条;仅 meta 行 `created` 字段值不同:旧包 `created` 落在 2026-10-18 13:57:00–17:31:11,提交包 `created` 落在 2026-10-18 18:21:56–21:12:22,均由 `run_experiments.py` 在组启动时刻写入,不影响任何 run 数据。)

三条结论:

1. 重构数值精确等价。67→102 文件只是把内联逻辑拆成脚本、补导出与 scan 数据,数值核心未动,同包同数据下输出逐比特相同。
2. 沙箱跨会话确定性可复现。同一四组在两次独立会话(不同容器、相隔约 4 小时)产出除时间戳外完全一致的结果,说明平台执行确定,既非随机,也没有环境漂移到数值层面。
3. 第 ② 项成立。公开 Replay 链接指向的会话 `944c87a6…` 确实完整执行了与提交物完全一致的代码包,不存在沙箱跑的子集与提交包脱节的问题。这是 10-18 复核发现并根治的。

> 说明:第六节的两批结果均产出于 SciencePal 沙箱的同一 Python 3.11 + numpy 2.3.0 / scipy 1.15.3 / sklearn 1.8.0 / pandas 2.3.3 镜像(与本地 §5.5 测定的沙箱同库版本栈一致),故两批之间逐比特一致。若换成另一套浮点环境(如本地默认栈),会触发 §5.5 已量化的约 60% 轨迹翻转,但组级统计量与文件级极值指纹仍在 §5.5 登记的带内。这与本节同环境逐比特一致的结论不矛盾,是不同对照层次下的两个结论。

七、定稿前的五组干净会话(2026-10-22)

第六节证明的是「同一个包在两个容器里给出同一批数」。这一节要证的是另一件事:**把分析脚本一起搬进沙箱、一行不改,能不能在沙箱里复算出版报告里那几个关键数字。**前面几轮的沙箱只执行实验(产出 jsonl),分析全部在本地做;评审完全可以问一句"那你们分析代码在别的机器上跑得起来吗"。所以定稿前补了五组会话,每次都开全新 thread、上传一个专用包、只跑一段给定指令、把事件流原样落盘。

7.1 五个会话

会话	thread	run	上传包(字节 / SHA256 前 8)	做了什么	终态
S3 跨环境重跑	`ffd303a0-37f4-47cf-a3ef-df63ad73133e`	`8bd03649…`	`failbound_crossenv_upload.zip`(13,749,286 / `2489aafb`)	样本外 2 基底 1680 run → β 扫描 1080 run → 线程敏感度探针	见 §7.3
S4-a 四个数字重算	`5c1597f6-ab12-4dff-b0dd-7801984d93cc`	`51260098…`	同包重建(13,753,352 / `aaeb9ab5`)	从原始 run 重算 AUC*/CI/最低增益、行序 ρ、贴下界计数、符号一致	completed
S4-b 诊断预算标定	`0930a437-879f-479c-98f9-b1983e1fa4de`	`8729393c…`	同 S4-a(13,753,352 / `aaeb9ab5`)	`probe_calibration.py` 全量:51 任务 × m∈{25,50,100,200} × 2 取点方式	completed
S5 准则卡裁决	`c6facfee-2d10-4ada-957d-d10ad01ac6d3`	`382bf45b…`	`failbound_card_upload.zip`(25,991 / `0dbdfeb6`,8 文件)	三段自然语言材料体系 → 读出三个数 → 拼命令行 → 原样回报	completed
S7 双 Agent 闭环	本地驱动(`crossenv/s7_loop.py`),调用 `d26e70b5` / `ba26fa43`	—	—	18 回合提议者 / 代码否决 / 终裁	completed

> S3 用的 `2489aafb` 与 S4 用的 `aaeb9ab5` 是两个字节数不同的构建(差 4,066 字节):后者补了 `holdout/data/tasks` 这个目录(见 §7.3 那个坑)。两者解包后的 23 个 `.py` 都过了「与提交包逐比特相同」的自查,差别只在数据目录占位。

五个会话的完整事件流已导出为离线 HTML(`demo/replay/s3|s4a|s4b|s5.html`)。**为什么导 HTML 而不只给 share 链接**:平台只把旧会话 `944c87a6…` 标了 `is_public`,新建会话不写进 `threads` 表;`POST /api/agents/{id}/publish` 返回403「API keys are limited to SDK agent, run and sandbox endpoints」,openapi 的 227 条路由里也没有任何 thread 公开端点。也就是说 API key 无权把新会话变公开,只能由登录用户在网页上点。评审视角不能依赖这个操作,所以事件流原样导出成 HTML 一并交付。

7.2 沙箱复算 vs 本地值(16 项,全部一致)

驱动脚本 `crossenv/cross_tools/steps.py`;沙箱值从各会话的最终交付里抄录(不是推断),本地值由 `crossenv/collect.py` 从产物 json 重新读一遍,两边拼表 → `results-extra/crossenv_repro.json`。

量	本地	沙箱	会话	布尔一致
AUC*	0.7307	0.7307	S4-a	一致:TRUE
95% CI	[0.5557, 0.8066]	[0.5557, 0.8066]	S4-a	一致:TRUE
最低增益	−33.9	−33.9	S4-a	一致:TRUE
`expt_gap` 行序 ρ	−0.187	−0.187	S4-a	一致:TRUE
前 400 行命中	0.168	0.168	S4-a	一致:TRUE
检查点数	8	8	S4-a	一致:TRUE
长度尺度贴下界(恰为 1e-5)	5	5	S4-a	一致:TRUE
长度尺度 ≤2e-5(报告口径)	7	7	S4-a	一致:TRUE

量	本地	沙箱	会话	布尔一致
打分场 IQR=0 的检查点	8	8	S4-a	一致:TRUE
`abs_gain` 符号一致	18/18	18/18	S4-a	一致:TRUE
25 点空间填充 ρ	0.042	0.042	S4-b	一致:TRUE
25 点空间填充决策一致率@0.6	0.449	0.449	S4-b	一致:TRUE
50 次随机抽样 ρ	0.758	0.758	S4-b	一致:TRUE
50 次随机抽样决策一致率@0.6	0.612	0.612	S4-b	一致:TRUE
标定任务数	51	51	S4-b	一致:TRUE
沙箱包 PY 文件与提交包逐比特相同	23/23	23/23	构建期自查	一致:TRUE

两条要说明的边界:

1. **一致的是数字,不是轨迹。** 第六节测过:换浮点环境时约六成 run 的 chosen 序列会翻转,

但组级统计量与极值指纹仍在带内。这一节的一致发生在**同一镜像内**(沙箱 numpy 2.3.0 栈),所以能做到逐位相同;它证明的是"分析代码可移植",不是"浮点无关"。

2. `steps.py` **用** `sys.executable` **而不是裸** `python`**。** 第一版在沙箱里报 `ModuleNotFoundError: numpy`

——机器上装了不止一个解释器,`python` 指到了没有依赖的那个。改成显式传当前解释器后一次通过。

7.3 S3:包搬得动,但有一个坑

S3 的第一步值得单独写,因为它是这次唯一一个**失败过的构建**:

- 第一版包解压后 `config._find_data_root()` 找不到 `holdout/data/tasks`,

于是静默回落到错误的 data root,样本外任务数变成 **0**——不报错,只是跑出空结果。修法是在 `build.py` 里显式补 `holdout/data/tasks`。

- 冻结包的 `config.py` 自带 `FAILBOUND_PROJECT` 开关,本来就是为"代码与数据不同级"准备的;

本方案需要**两套数据根**(冻结臂 `$PWD`、样本外臂 `$PWD/holdout`),正好用这个开关切换,不去改包内任何一行。

- 构建末做了一次自查:解出来的 **23/23 个** `.py` **与提交包逐比特相同。**

这条比"我改过配置"更有说服力——分析脚本一行没动,所以复算结果才有意义。

S3 样本外重跑已于打包前中止,作为赛后独立复算处理,未计入本次平台提交。 中止原因为沙箱时长限制,样本外 1680 run 未在本平台会话内完成。该发现已登记为可复用的错误恢复经验。本次提交以 S4-a / S4-b / S5 / S7 四个已完成会话与主 Replay `944c87a6` 为准,其 16 项跨环境对照全部一致(16 项不含样本外;S3 样本外重跑已中止,未计入)。

7.4 S7 为什么开新会话而不是复用

S7 要测的是"提议者与终裁互不知道对方看到了什么",复用旧会话会让上下文泄漏。所以提议者(`d26e70b5`)与终裁(`ba26fa43`)各自独立调用,每回合 2 次调用、18 回合、平均 50 秒/回合,真值只用于打分,从不进 prompt。